

Adaptive Continuous Authentication & Account-Takeover Detection

Real-Time Keystroke Dynamics & Behavioral Anomaly Detection for Zero-Trust Session Security

Submitted by:	Department:
Yatharth Kansal (Roll No: 1024030456) <i>ML & Core Engine Lead</i>	Computer Science & Engineering TIET Patiala
Anjali Kumari (Roll No: 1024030457) <i>Frontend & Integration Lead</i>	Computer Science & Engineering TIET Patiala
Ridhi Batra (Roll No: 1024030464) <i>Testing & Documentation Lead</i>	Computer Science & Engineering TIET Patiala

Submitted to:
Prof. Asif (Dr. Jeelani Asif)
Department of Computer Science and Engineering

Chapter 1: Introduction

1.1 Background and Problem Formulation

Modern enterprise access security is predominantly built around **point-in-time perimeter authentication**. Users supply credentials—such as passwords, time-based one-time codes (TOTP), or hardware security keys—exclusively at the time of session creation. Once verified, the authorization state remains static, treating the active session token as perpetually trusted until explicit timeout or manual logout.

This architectural paradigm gives rise to severe post-authentication attack vectors:

- **Unattended Workstation Hijacking:** An employee who leaves their laptop unlocked permits any physically proximate individual to impersonate them instantly.
- **Session Token Exfiltration:** Malicious infostealers or Cross-Site Scripting (XSS) attacks steal active cookies, completely bypassing login credentials.
- **Perimeter Blindness:** Conventional authorization engines possess zero visibility into mid-session anomalies and cannot modulate trust dynamically.

1.2 Project Objectives

The **Adaptive Continuous Authentication (ACA)** framework eliminates these blindspots by constructing a passive, Zero-Trust biometric defense layer. Our core engineering goals comprise:

- **Transparent Verification:** Evaluating user identity through micro-rhythmic keystroke dynamics (dwell time, flight duration, and cadence) without prompting the user or disrupting workflow.
- **Zero-Keystroke-Logging Privacy:** Guaranteeing that typed text is never stored or transmitted, using client-side HMAC-SHA256 pseudonymization.
- **Continuous Trust Scoring:** Mapping 5-dimensional behavioral vectors to a continuous Trust Score (0–100%) updated across sliding 10-second temporal windows.
- **Graduated Policy Responses:** Formulating an automated 3-tier policy engine: Transparent Allow ($\geq 70\%$), Prompt Re-authentication (40–69%), and Restrict / Lock Session ($< 40\%$).
- **Inactive Window Leniency:** Implementing a holding state for idle windows (< 5 keystrokes) so cognitive pauses never penalize legitimate users.

Chapter 2: Methodology and Architecture

2.1 High-Level Architecture

The ACA platform decouples low-level event capture, privacy hashing, biometric feature extraction, machine learning inference, and security operations visualization across a five-stage modular pipeline:

1. **Capture Subsystem:** Captures millisecond-accurate keydown and keyup events via monotonic OS clocks (pynput) or browser DOM APIs.
2. **Cryptographic Anonymization:** Converts raw key codes into irreversible HMAC-SHA256 hashes using per-session cryptographic keys. Only hashed IDs and timings leave the browser.
3. **Sliding Window Feature Extractor:** Discretizes the event stream into 10-second windows and extracts Dwell (Hold) time, Flight time, Typing Cadence, Jitter, and Pause counts.

4. **Scikit-Learn Anomaly Pipeline:** Imputes missing values with medians, standardizes feature variance, and computes outlier scores using an ensemble of 100 Isolation Trees.
5. **Trust Scorer & Dynamic Policy:** Maps raw decision values to a continuous 0–100% score and updates the dark-mode SOC web dashboard in real time.

2.2 Biometric Feature Formulation

Within each 10-second behavioral window, the engine synthesizes five primary biometric indicators:

- **Dwell (Hold) Time (ms):** Duration a physical key remains pressed: $T_{hold} = t_{release} - t_{press}$.
- **Flight Time (ms):** Transition interval between releasing one character and pressing the next: $T_{flight} = t_{press}(n+1) - t_{release}(n)$.
- **Typing Cadence:** Rate of character generation normalized over window duration: $CPS = Count / 10.0s$.
- **Hold Jitter (Std Dev):** Intra-individual variance in hold duration representing motor consistency.
- **Pause Frequency:** Total occurrences of hesitation intervals exceeding 500 ms.

2.3 Machine Learning Pipeline & Trust Mapping

We utilize an unsupervised **Isolation Forest** (`n_estimators=100`, `contamination='auto'`) encapsulated in a Scikit-Learn Pipeline preceded by `SimpleImputer(strategy='median')` and `StandardScaler`. Because continuous authentication operates under a one-class paradigm—where genuine user data is available, but future attackers are unobserved—Isolation Forest explicitly isolates abnormal points with minimal partition splits.

The model produces an anomaly score $s \in [-0.10, +0.10]$, linearly mapped and clipped into a 0–100% Trust Score:

Trust Score = clip($50.0 + (s / 0.10) * 50.0$, 0.0, 100.0)

Chapter 3: Results and Evaluation

3.1 Engineering Performance Benchmarks

Empirical benchmarks demonstrate that the continuous authentication pipeline operates with negligible overhead and sub-millisecond evaluation latency, fulfilling all real-time requirements:

Performance Metric	Specification Target	Measured Result	Compliance Status
Model Inference Latency	≤ 5.0 ms	0.42 – 0.88 ms	Passed (Sub-ms)
End-to-End API Roundtrip	≤ 50.0 ms	12.3 – 24.1 ms	Passed (Real-Time)
Evaluation Window Size	10.0 seconds	10.0 s fixed	Fully Compliant
Active Keystroke Gate	≥ 5 characters	5 char presses	Fully Compliant
HMAC Cryptographic Cost	≤ 1.0 ms / key	0.08 ms / key	Passed (Negligible)
Server Memory Footprint	≤ 100 MB	38.4 MB	Passed (Lightweight)
Automated Test Pass Rate	100% Pass	11 / 11 (100%)	100% Success

3.2 Empirical Baseline & Speed Analysis

A critical finding emerged during initial live testing: fluent, natural typing (3.5–5.5 chars/sec, 60–75 ms hold, 120 ms flight) resulted in low trust scores (~48.3%, Verify). Diagnostic analysis revealed that the initial baseline (session U01/S02) was recorded under slow, cautious typing conditions (0.5–2.6 chars/sec, 100–124 ms dwell, frequent pauses). Because the Isolation Forest boundary was fitted exclusively to the 6 slow windows, normal-tempo typing was correctly identified as an outlier relative to that specific baseline.

When evaluated against S02-tempo slow typing, the model immediately produced a score of **+0.0960 (98.0% Trust, ALLOW)**. This verified that the anomaly detection engine is mathematically rigorous, and highlighted the operational requirement to collect broader multi-tempo enrollment baselines.

Chapter 4: Conclusion

4.1 Summary of Deliverables

The mid-semester prototype successfully realizes an end-to-end continuous authentication architecture:

- Hardware & Browser Data Collector:** HMAC-SHA256 privacy-preserving event recording.
- Biometric Feature Pipeline:** Dwell, flight, cadence, and pause feature synthesis.
- Machine Learning Core:** Isolation Forest anomaly detector with continuous linear trust scoring.
- Operations Dashboard:** Live dark-mode interface featuring dynamic SVG gauges and policy state enforcement.
- Automated Test Suite:** 11 comprehensive unit and integration tests passing with 100% success.

4.2 Next Steps & Roadmap

- **Multi-Session Enrollment Expansion:** Recording additional natural-tempo sessions (S03, S04) to expand baseline coverage to 60+ windows.
- **Adaptive Baseline Retuning:** Implementing controlled incremental retraining triggered only after successful primary re-authentication, preventing adversarial poisoning.
- **Multi-Modal Sensor Fusion:** Incorporating mouse dynamics (speed, acceleration, curvature) alongside keystrokes for fused risk estimation.